



Evolutionary Computation

Assignment 1: Greedy Heuristics

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Problem	1
2	Pseudocode	2
2.1	Objective function	2
2.2	Random	2
2.3	Nearest Neighbor	3
2.4	Nearest Neighbor Flexible	4
2.5	Greedy Cycle	5
3	Results	6
4	Conclusion	10

1 Problem

We are provided with a dataset consisting of three integer columns, where each row represents a node. The first two columns specify the x and y coordinates of each node in a two-dimensional plane, while the third column indicates the cost associated with each node.

The objective is to choose exactly half of the nodes (rounding up if the total number is odd) and construct a Hamiltonian cycle, a closed loop that visits each selected node exactly once. The aim is to minimize the combined total of the path length and the cumulative cost of the selected nodes.

For this assignment, we had to implement three algorithms:

- Random solution.
- Nearest neighbor considering adding the node only at the end of the current path.
- Nearest neighbor considering adding the node at all possible position, i.e. at the end, at the beginning, or at any place inside the current path.
- Greedy cycle

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory/tree/main/assignment_1

2 Pseudocode

In this section, we will present the pseudocodes of the algorithms. For this assignment, we will code in C++.

2.1 Objective function

Our objective function passed correctly the solution checker tests for TSPA and TSPB.

Objective Function

```
computeObjective(path, distance matrix, nodes) :  
    totalDist  $\leftarrow$  0  
    totalCost  $\leftarrow$  0  
    FOR i FROM 0 TO LENGTH(path) - 2 DO :  
        totalCost  $\leftarrow$  totalCost + nodes[path[i]].cost  
        totalDist  $\leftarrow$  totalDist + dist[path[i]][path[i+1]]  
  
    RETURN totalDist + totalCost
```

2.2 Random

Random Solution

```
randomSolution(selectedNodes) :  
    path  $\leftarrow$  COPY(selectedNodes)  
    shuffle(path)  
    APPEND path[0] TO path // return to start  
    RETURN path
```

2.3 Nearest Neighbor

Nearest Neighbor

nearestNeighborEnd(selectedNodes, distance matrix, nodes, starting node):

```
path ← [startNodeId]
maxSize ← FLOOR(LENGTH(nodes) / 2)
visited ← array of FALSE with size LENGTH(nodes)
visited[startNodeId] ← TRUE
WHILE LENGTH(path) < maxSize DO
    bestNode ← -1
    bestScore ← INF
    FOR each node IN nodes DO
        IF visited[node.id] THEN CONTINUE
        score ← dist[path[-1]][node.id] + node.cost
        IF score < bestScore THEN
            bestScore ← score
            bestNode ← node.id
        END IF
    END FOR
    APPEND bestNode TO path
    visited[bestNode] ← TRUE
END WHILE
APPEND startNodeId TO path
RETURN path
```

2.4 Nearest Neighbor Flexible

Nearest Neighbor Flexible Heuristic

```

path ← [startNodeId]
maxSize ← FLOOR(LENGTH(nodes) / 2)
visited ← array of FALSE with size LENGTH(nodes)
visited[startNodeId] ← TRUE
WHILE LENGTH(path) < maxSize DO
  bestNode ← -1
  bestPos ← -1
  bestScore ← INF
  FOR each node IN nodes DO
    IF visited[node.id] THEN CONTINUE
    FOR i FROM 0 TO LENGTH(path) DO
      tempPath ← COPY(path)
      INSERT node.id AT position i IN tempPath
      score ← computeObjective(tempPath, dist, nodes)
      IF score < bestScore THEN
        bestScore ← score
        bestNode ← node.id
        bestPos ← i
      END IF
    END FOR
  END FOR
  INSERT bestNode AT bestPos IN path
  visited[bestNode] ← TRUE
END WHILE
APPEND path[0] TO path
RETURN path

```

2.5 Greedy Cycle

Greedy Cycle

```

path ← [startNodeId]
numToSelect ← FLOOR(LENGTH(nodes) / 2)
visited ← array of FALSE with size LENGTH(nodes)
visited[startNodeId] ← TRUE
// Select best second node to form initial cycle
bestSecondNode ← -1
bestInitialScore ← INF
FOR each node IN nodes DO
  IF visited[node.id] THEN CONTINUE
  score ← dist[startNodeId][node.id] + nodes[startNodeId].cost + node.cost
  IF score < bestInitialScore THEN
    bestInitialScore ← score
    bestSecondNode ← node.id
  END IF
END FOR
APPEND bestSecondNode TO path
visited[bestSecondNode] ← TRUE
APPEND startNodeId TO path // complete initial 2-node cycle
// Iteratively insert remaining nodes
WHILE LENGTH(path) < numToSelect + 1 DO
  bestNode ← -1
  bestPos ← -1
  bestScore ← INF
  FOR each node IN nodes DO
    IF visited[node.id] THEN CONTINUE
    FOR i FROM 1 TO LENGTH(path) - 1 DO
      tempPath ← COPY(path)
      INSERT node.id AT position i IN tempPath
      score ← computeObjective(tempPath, dist, nodes)
      IF score < bestScore THEN
        bestScore ← score
        bestNode ← node.id
        bestPos ← i
      END IF
    END FOR
  END FOR
  INSERT bestNode AT bestPos IN path
  visited[bestNode] ← TRUE
END WHILE
RETURN path

```

3 Results

After trying these different algorithms, we will check their effectiveness by showing the constructed cycle.

In order, we will start from the algorithm that seems least efficient to the one that shows the best results. In the plot, the cost of each node is represented using a color scale: yellow indicates higher cost, while purple indicates lower cost. Additionally, the edges show the sequence of nodes in the path starting from 0. Below the figure, we displayed the IDs of the nodes in the best path.

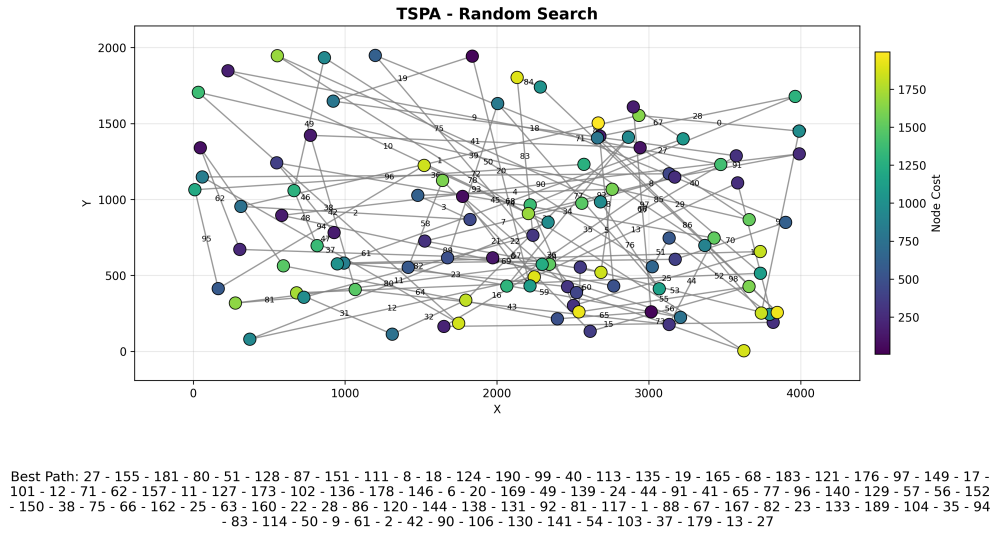


Figure 1: Random Search TSPA

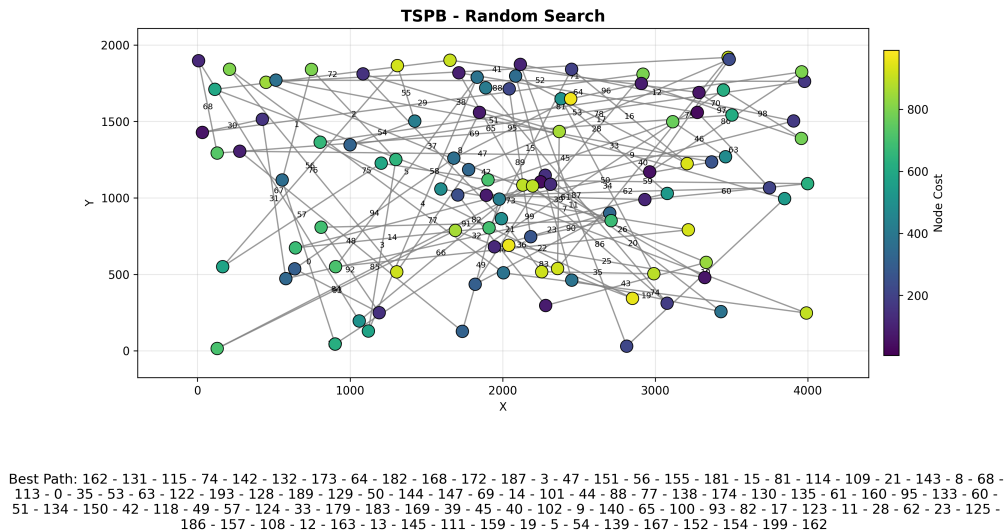


Figure 2: Random Search TSPB

For the Random Search algorithm, we quickly realize its inefficiency because the cycle is cut everywhere and there is no cost optimization as we can see many yellow nodes.

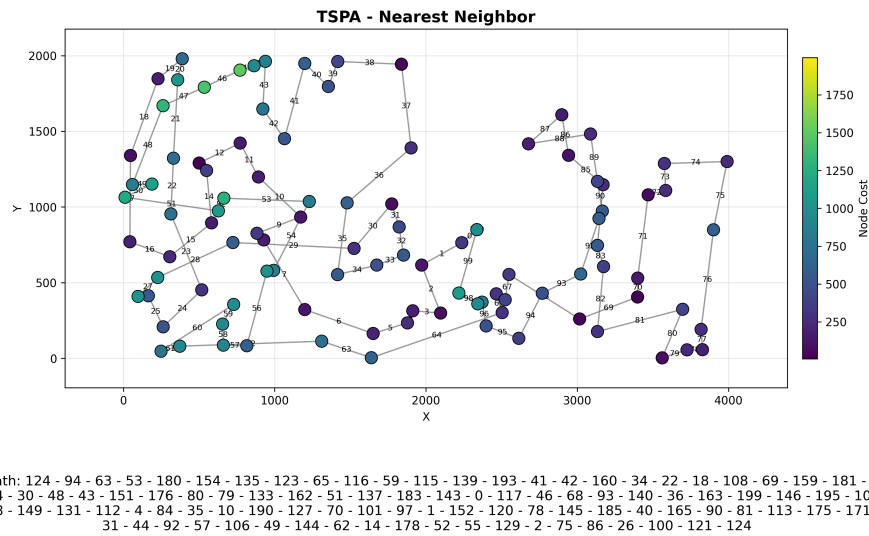


Figure 3: Nearest Neighbor TSPA (End)

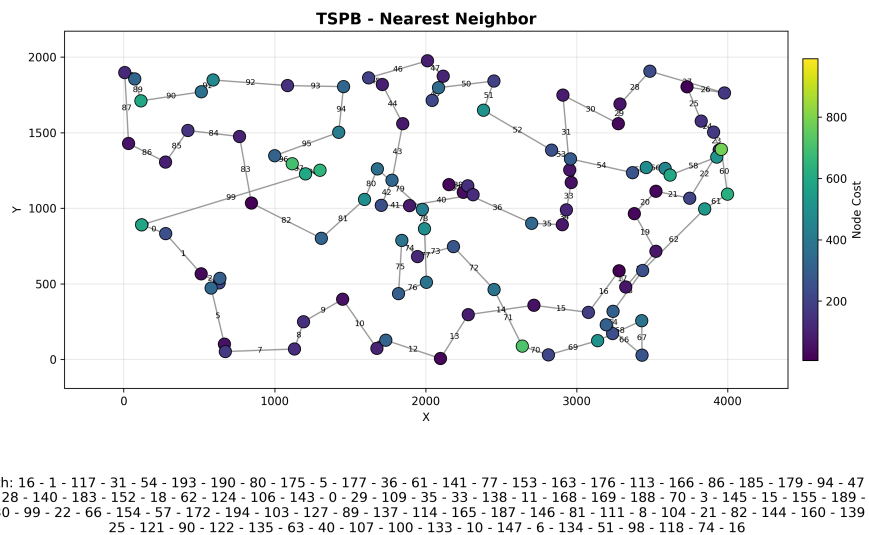


Figure 4: Nearest Neighbor TSPB (End)

For the Nearest Neighbor algorithm, which considers only the end possible positions when constructing the path to the next node, we can also observe its inefficiency. Although it performs much better than Random Search what we can be seen with cost optimization by more purple/blue nodes, the paths are also crossed and appear quite messy in some parts.

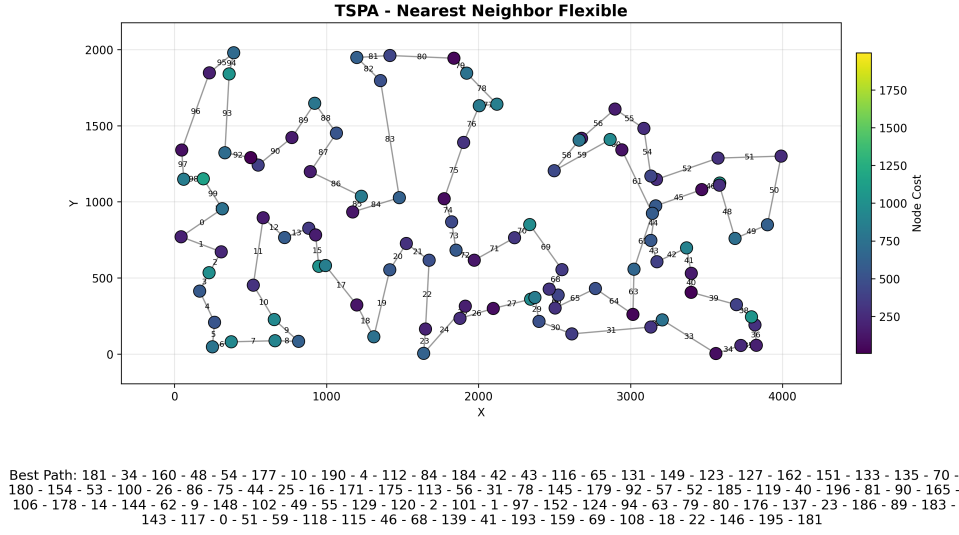


Figure 5: Nearest Neighbor Flexible TSPA

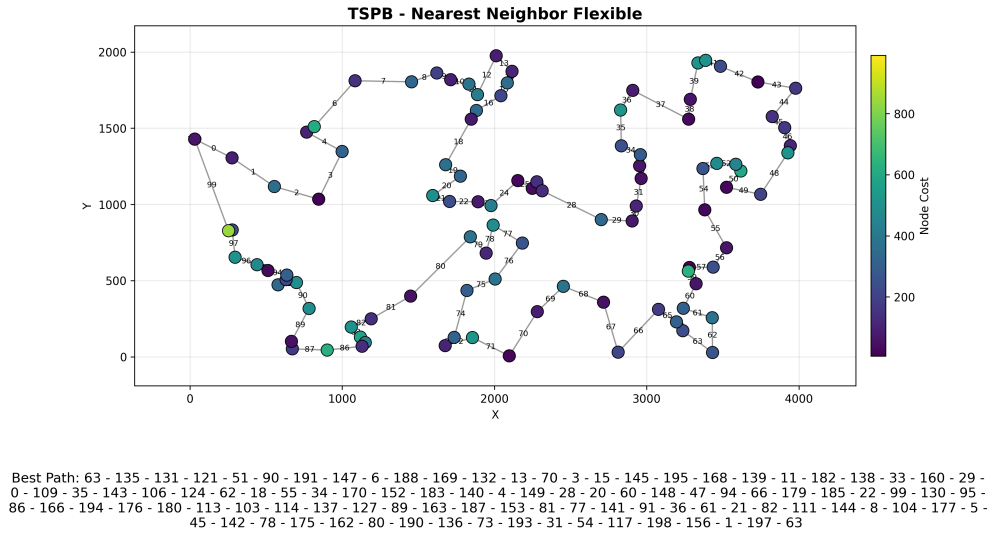
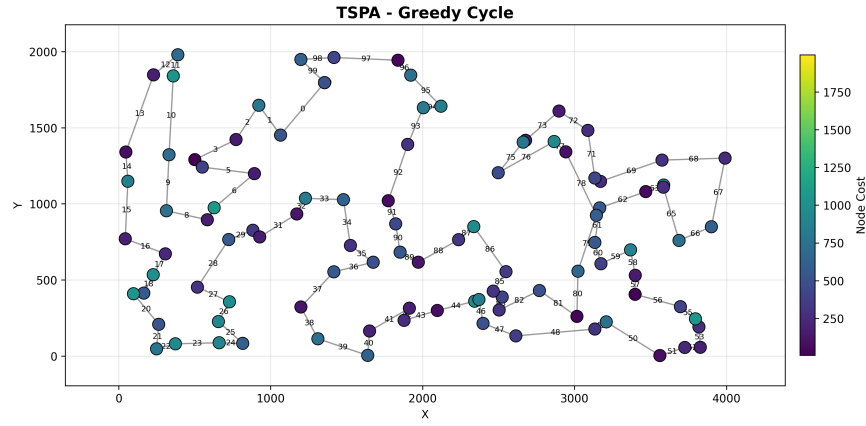


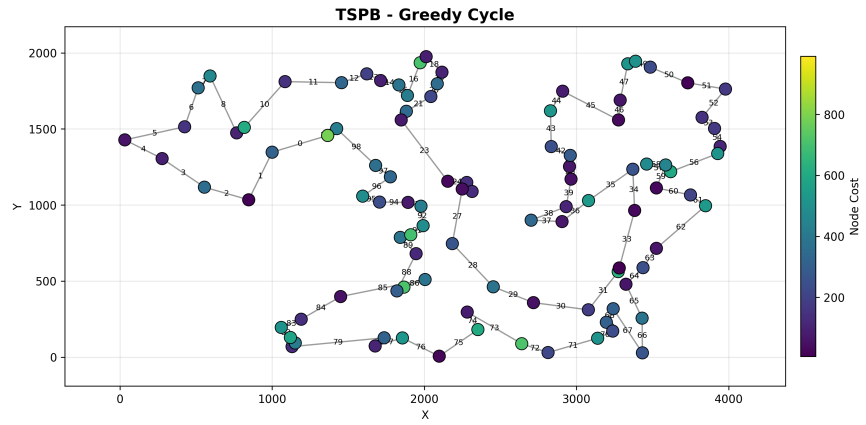
Figure 6: Nearest Neighbor Flexible TSPB

For the Nearest Neighbor Flexible algorithm, where the next node can be added at any position in the existing path, the results are quite good. Cost optimization is achieved through the inclusion of low cost nodes, and the paths are less crossed, making them appear more natural.



Best Path: 0 - 46 - 68 - 139 - 193 - 41 - 115 - 5 - 42 - 181 - 159 - 69 - 108 - 18 - 22 - 146 - 34 - 160 - 48 - 54 - 30 - 177 - 10 - 190 - 4 - 112 - 84 - 35 - 184 - 43 - 116 - 65 - 59 - 118 - 51 - 151 - 133 - 162 - 123 - 127 - 70 - 135 - 180 - 154 - 53 - 100 - 26 - 86 - 75 - 44 - 25 - 16 - 171 - 175 - 113 - 56 - 31 - 78 - 145 - 179 - 92 - 57 - 52 - 185 - 119 - 40 - 196 - 81 - 90 - 165 - 106 - 178 - 14 - 144 - 62 - 9 - 148 - 102 - 49 - 55 - 129 - 120 - 2 - 101 - 1 - 97 - 152 - 124 - 94 - 63 - 79 - 80 - 176 - 137 - 23 - 186 - 89 - 183 - 143 - 117 - 0

Figure 7: Greedy Cycle TSPA



Best Path: 85 - 51 - 121 - 131 - 135 - 63 - 122 - 133 - 10 - 90 - 191 - 147 - 6 - 188 - 169 - 132 - 13 - 161 - 70 - 3 - 15 - 145 - 195 - 168 - 29 - 109 - 35 - 0 - 111 - 81 - 153 - 163 - 180 - 176 - 86 - 95 - 128 - 106 - 143 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 183 - 140 - 4 - 149 - 28 - 20 - 60 - 148 - 47 - 94 - 66 - 22 - 130 - 99 - 185 - 179 - 172 - 168 - 194 - 113 - 114 - 137 - 103 - 89 - 127 - 165 - 187 - 146 - 77 - 97 - 141 - 91 - 36 - 61 - 175 - 78 - 142 - 45 - 5 - 177 - 82 - 87 - 21 - 8 - 104 - 56 - 144 - 160 - 33 - 138 - 182 - 11 - 139 - 134 - 85

Figure 8: Greedy Cycle TSPB

For the Greedy Cycle algorithm, the results are very similar to those of the Nearest Neighbor Flexible algorithm. Additionally, we can see that almost all nodes have low costs and the paths are not crossed, so they are forming a more natural cycle.

Method	Avg (Min, Max)
Random Search	264585.05 (235867, 293888)
Nearest Neighbor End	85108.51 (83182, 89433)
Nearest Neighbor Flexible	73594.10 (71868, 75953)
Greedy Cycle	72635.98 (71488, 74410)

Table 1: Average, minimum, and maximum objective values for TSPA

Method	Avg (Min, Max)
Random Search	212969.54 (193015, 236193)
Nearest Neighbor End	54390.43 (52319, 59030)
Nearest Neighbor Flexible	48694.25 (44609, 57283)
Greedy Cycle	51400.60 (49001, 57324)

Table 2: Average, minimum, and maximum objective values for TSPB

Thanks to these tables, we can see that the Greedy Cycle and Nearest Neighbor Flexible algorithms are the most effective methods. The higher scores depend on the type of scenario for the TSP problem. These two methods achieved almost the same maximum value during the experiment, while the lowest value varied depending on the specific problem.

4 Conclusion

In this study, we compared four algorithms for constructing TSP cycles: Random Search, Nearest Neighbor, Nearest Neighbor Flexible, and Greedy Cycle. The Random Search method produced the least effective results, generating highly crossed paths with little cost optimization. The standard Nearest Neighbor algorithm improved upon Random Search but still produced somewhat disorganized paths in certain areas.

The Nearest Neighbor Flexible algorithm generated more natural, less crossed paths and achieved better cost optimization. Similarly, the Greedy Cycle algorithm produced results very close to Nearest Neighbor Flexible, including most of the low-cost nodes and forming well-structured cycles.

The difference between these two methods may arise from the way they select the next node. If one algorithm finds a slightly better node early in the process and follows that path, it might miss the better local optimal route, since it only evaluates the best next node based on a limited look-ahead.